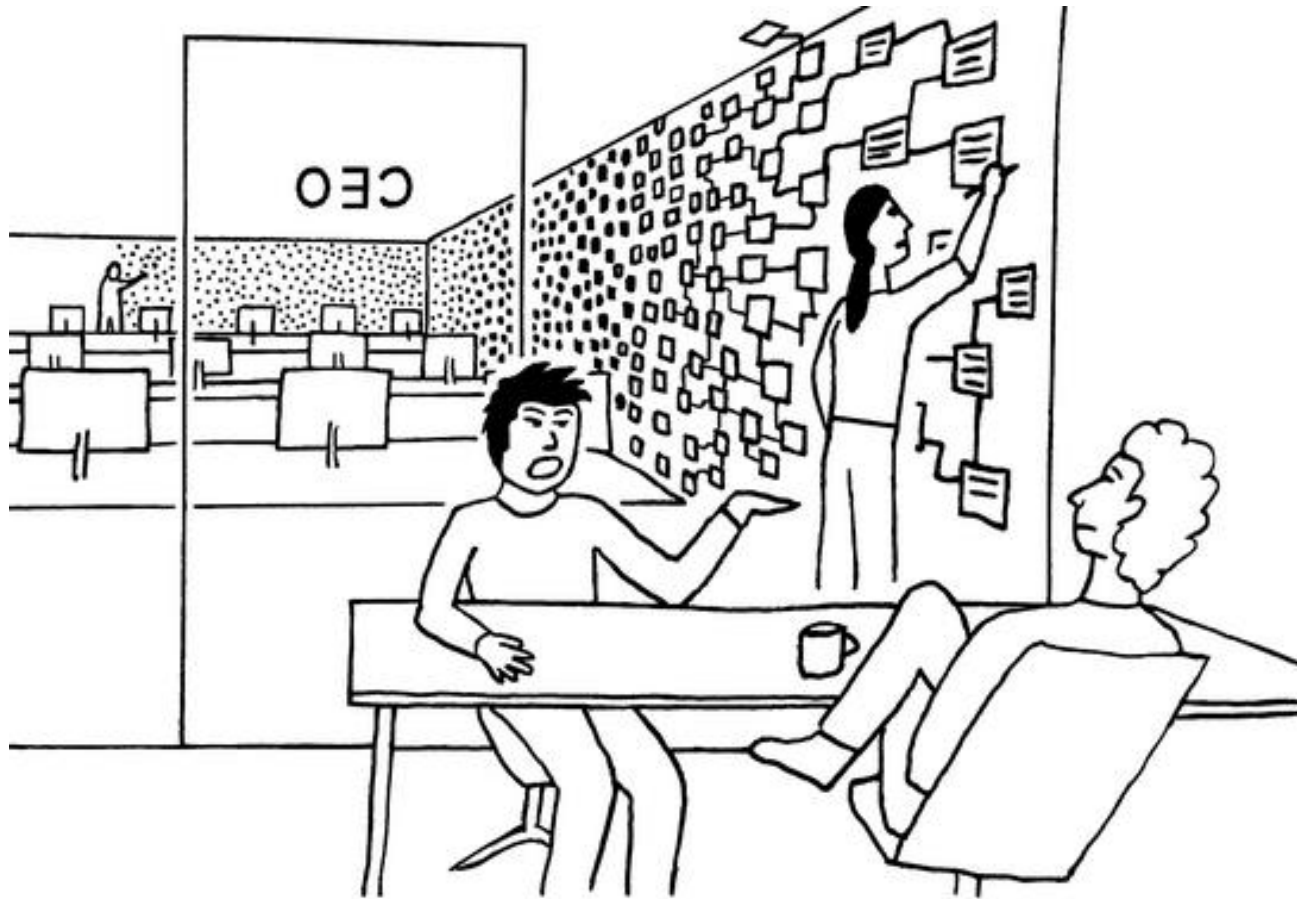


CS2300: File Structures and Introduction to Database Systems

Lecture 5: Relational Model and Relational Database Constraints

San Yeung



Piotr@Dataedo

HEY RICHARD. REMEMBER WHEN YOU ASKED US
TO MAP OUR SYSTEM?
WELL, WE'RE GOING TO NEED A BIGGER OFFICE...

Data Models



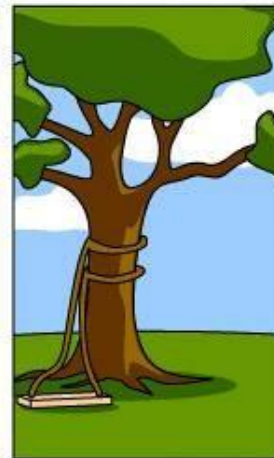
How the customer explained it



How the Project Leader understood it



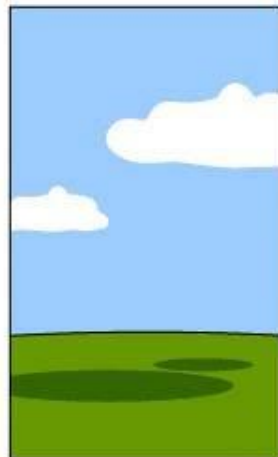
How the Analyst designed it



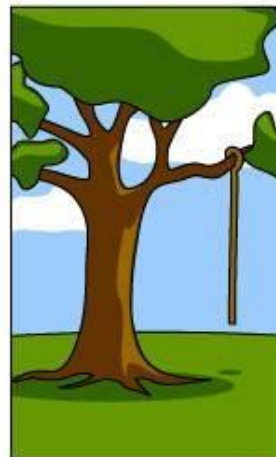
How the Programmer wrote it



How the Business Consultant described it



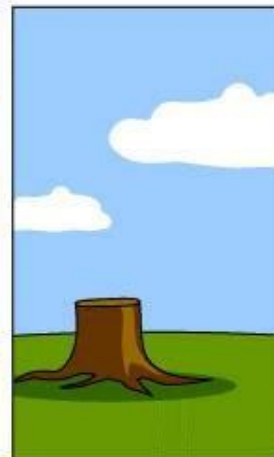
How the project was documented



What operations installed



How the customer was billed



How it was supported



What the customer really needed

Data Models, Schemas, and Instances

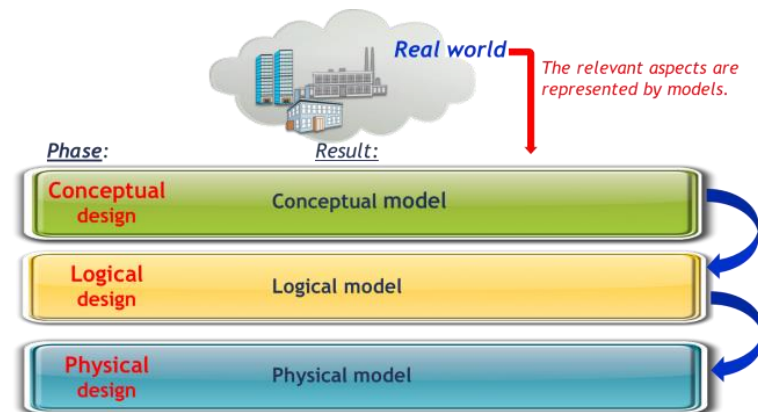
- Wayback Machine has 2 PB + 20 TB/month (2006)
- NOAA has ~1 PB climate data (2007)
- CERN's LHC will generate 15 PB a year (2008)
- Google processes 20 PB a day (2008)
- "all words ever spoken by human beings" ~ 5 EB
- <https://www.youtube.com/watch?v=N7YLHYtUbiE>



640K ought to
be enough for
anybody.

Representational Data Models

- Also called **logical** data models
- Provides high-level design concepts in as much detail as possible, without regard to how the data will be physically *implemented/stored* in the database
- Typical features:
 - Define attributes for each entity
 - Define primary and secondary keys
 - Define relationships between entities
 - Intended for a more technical audience
 - Basis for application development and physical design



The screenshot shows the Microsoft SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the 'Databases' folder expanded, showing 'AdventureWorks2000' and 'AdventureWorksDW'. The 'Tables' folder is also expanded, showing tables like 'dbo.authors', 'dbo.cust_demo', 'dbo.discounts', 'dbo.employee', 'dbo.jobs', 'dbo.pub_info', 'dbo.publishers', 'dbo.roysched', 'dbo.sales', 'dbo.stores', 'dbo.titleauthor', and 'dbo.titles'. The 'Views' folder is also expanded, showing 'System Views', 'Synonyms', 'Programmability', 'Service Broker', 'Storage', and 'Security'.

The main window displays a query plan for a query titled 'LEVIN\LEVIN_2... - dbo.View_1*'. The query plan shows a nested loop join between the 'authors' and 'publishers' tables, followed by a join with the 'titles' table. The query text is visible at the bottom of the window.

Query Text:

```
SELECT TOP (100) PERCENT dbo.authors.au_name, dbo.authors.au_fname, dbo.publishers.pub_name,
dbo.titles.title, dbo.titles.type, dbo.titles.price
FROM
  dbo.authors INNER JOIN
  dbo.titleauthor ON dbo.authors.au_id = dbo.titleauthor.au_id INNER JOIN
  dbo.titles ON dbo.titleauthor.title_id = dbo.titles.title_id INNER JOIN
  dbo.publishers ON dbo.titles.pub_id = dbo.publishers.pub_id
WHERE (dbo.authors.state = 'CA' OR
```

The query plan shows a nested loop join between the 'authors' and 'publishers' tables, followed by a join with the 'titles' table. The query plan is visualized with a diamond symbol representing the join operation. The 'authors' table is joined to the 'publishers' table, and the result is then joined to the 'titles' table. The query plan is visualized with a diamond symbol representing the join operation.

- Data is organized into tables or relations.
- Relationships between tables are represented using primary and foreign keys.
- Widely used and is fundamental in database design.
- Backbone of most RDBMS: MySQL, Oracle, PostgreSQL, Microsoft SQL Server...

Example: Object- Relational Model

- Hybrid of the relational and object-oriented data models
- Database can store complex data structures like arrays, objects, and nested records.
- Object-relational features is well supported by PostgreSQL and Oracle.

Object Type <i>person_typ</i>	
Attributes	Methods
idno first_name last_name email phone	get_idno display_details

Object	
idno:	65
first_name:	Verna
last_name:	Mills
email:	vmills@example.com
phone:	1-650-555-0125

Object	
idno:	101
first_name:	John
last_name:	Smith
email:	jsmith@example.com
phone:	1-650-555-0135

```
CREATE TYPE person_typ AS OBJECT (
  idno          NUMBER,
  first_name    VARCHAR2(20),
  last_name     VARCHAR2(25),
  email         VARCHAR2(25),
  phone         VARCHAR2(20),
  MAP MEMBER FUNCTION get_idno RETURN NUMBER,
  MEMBER PROCEDURE display_details ( SELF IN OUT NOCOPY person_typ ));
```



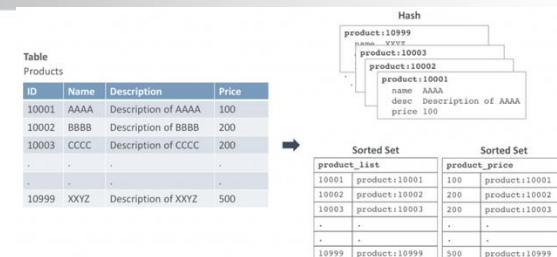
```
-- requires existing person_typ fr. Ex 1-1
CREATE TABLE contacts (
  contact      person_typ,
  contact_date DATE );

INSERT INTO contacts VALUES (
  person_typ (65, 'Verna', 'Mills', 'vmills@example.com', '1-650-555-0125'),
  to_date('24 Jun 2003', 'dd Mon YYYY'));
```



Example: NoSQL Data Models

- **Document-based:** Stores data as documents instead of using a fixed schema, often in formats like JSON or BSON. Ex) MongoDB
- **Column-based:** Organizes and stores data into columns rather than rows. Best for read-heavy applications. Ex) Apache Cassandra, Google Bigtable, and HBase.
- **Key-Value stores:** Each item is stored as a key-value pair. Simplest and high-speed for key-value-based operations such as user session management. Ex) Redis (in-memory db) and Amazon DynamoDB.
- **Graph-based:** Focus on the relationships between data points. Used for social networks, recommendation engines, etc. Ex) Neo4j and OrientDB.



Row-oriented

ID	Name	Grade	GPA
001	John	Senior	4.00
002	Karen	Freshman	3.67
003	Bill	Junior	3.33

Column-oriented

Name	ID	Grade	ID	GPA	ID
John	001	Senior	001	4.00	001
Karen	002	Freshman	002	3.67	002
Bill	003	Junior	003	3.33	003

```
{
  _id: <ObjectId>,
  username: "123xyz",
  contact: {
    phone: "123-456-7890",
    email: "xyz@example.com"
  },
  access: {
    level: 5,
    group: "dev"
  }
}
```

Embedded sub-document

Embedded sub-document

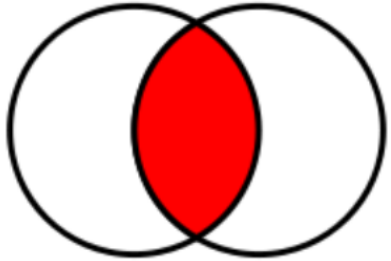
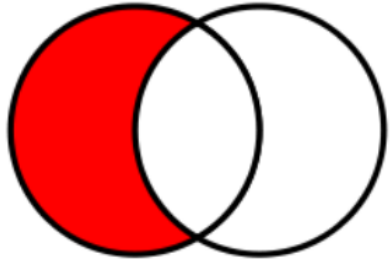
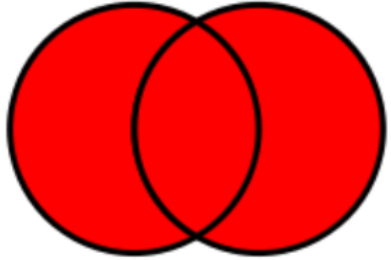
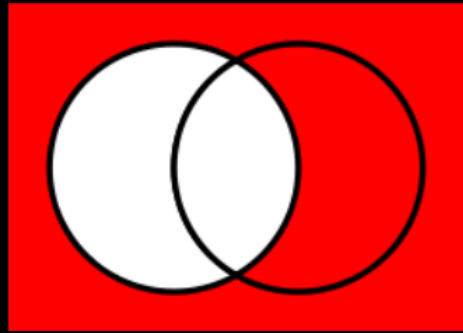
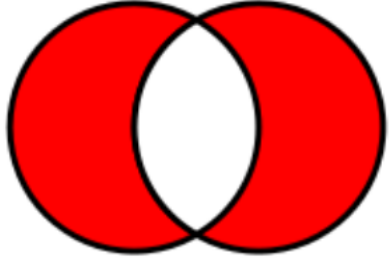
Chapter Outline

- Relational Model Concepts
- Relational Model Constraints
- Update Operations &
Dealing with Constraint Violations



Introduction

- The model was first proposed by Dr. E.F. Codd of IBM Research in 1970 in the following paper:
 - "A Relational Model for Large Shared Data Banks," **Communications of the ACM**, June 1970
- The paper caused a major revolution in the field of database management due to its simplicity and mathematical foundation
 - *uses the concept of a mathematical relation as its basic building block*
 - *lays its theoretical basis in set theory and predicate logic*

	$A = B \cap C$		$A = B \setminus C$
	$A = B \cup C$		$A = \neg B$
	$A = B \Delta C$	A Not A (or $\neg A$)	

Set Theory

Relational Model Concepts



- The relational model represents the database as a **collection** of *relations*
- Each relation, informally, resembles a table of values or a *flat file* of records
- Each **row** represents a real-world entity, relationship included
- All values in a **column** are of the same data type

Diagram illustrating the structure of a relation (table):

Relation Name: STUDENT

Attributes: Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa

Tuples (rows):

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

Cardinality = # of rows/tuples

Arity/Degree = # of columns/attributes



Domains, Attributes, Tuples, and Relations

- A **domain** D is a set of atomic values specified on a data type from which the data values are drawn

Examples: (domain name, range)

1. **USA_phone_numbers:**
 1. The set of ten-digit phone numbers valid in the United States
2. **Names:**
 1. The set of character strings that represent names of persons
3. **Grade_point_averages:**
 1. Possible values of computed GPAs; each must be a real (floating-point) number between 0 and 4



Domains, Attributes, Tuples, and Relations

- A **relation schema** R is denoted by $R(A_1, A_2, \dots, A_n)$
 - A_i is an attribute of R
 - The **domain** of A_i is denoted by **dom**(A_i)
- **Example:** (*university students*)

STUDENT (Name, Ssn, Home_phone, Address, Age, Gpa)

With data type:

STUDENT (Name:string, Ssn:string, Home_phone:string, Address:string, Age:integer, Gpa:real)

- $\text{dom}(\text{Name}) = \text{Names}$
- $\text{dom}(\text{Home_phone}) = \text{USA_phone_numbers}$
- $\text{dom}(\text{Gpa}) = \text{Grade_point_averages}$



Domains, Attributes, Tuples, and Relations

- A **relation** (or **relation state**) of the schema $R (R=(A_1, A_2, \dots, A_n))$ is a set of n -tuples denoted by $r(R) = \{t_1, t_2, \dots, t_m\}$.
- Each tuple t is an ordered list of n values,
 $t = \langle v_1, v_2, \dots, v_n \rangle$, where each value is an element of $\text{dom}(A_i)$ or is a special NULL value
- **Example:**

CUSTOMER (Cust_id:string, Cust_name:string, Address:string, Phone#:string)

$\langle \text{"632895"}, \text{"John Smith"}, \text{"101 Main St. Atlanta, GA 30332"}, \text{"(404) 894-2000"} \rangle$

Figure 5.6 One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

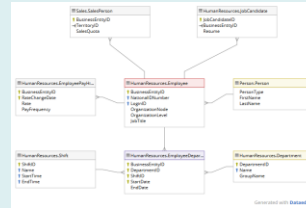
<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Schemas, Instances, and Database State

In a data model, there are two distinguishable components

Database Description

Database schema



The Actual Database

Data/record instances

Year	New Record Instances	Percent of Dataset
2004	928	0.1%
2005	43,425	3.6%
2006	33,899	2.8%
2007	31,053	2.6%
2008	25,138	2.1%
2009	88,580	7.4%
2010	179,498	15.0%
2011	188,810	15.8%
2012	248,439	20.8%
2013	292,342	24.5%
2014	61,695	5.2%

Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

The structure of each record type is clearly shown

Database Schema

- The ***description*** of a database
 - is it going to be changed frequently?
- Includes descriptions of the database structure, data types, and **the constraints** on the database.
- Schema Diagram:
 - An ***illustrative*** display of (most aspects of) a database schema.
 - Supported by most data models

Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

Each schema object can be referred to as a **record type** or a **schema construct**

The structure of each record type is clearly shown

Database State and Instances

- The actual data in a database may change frequently
- **Database state/snapshot:** The actual data stored in a database at a *particular moment in time*
- In a given state, each schema construct has its own set of **instances**
 - E.g. the **STUDENT** construct will contain the set of individual student entities (records) as its instances

Example of a Database State

State or
instances?

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2

A database that stores student and course information.

State or
instances?

Database State and Instances

- *When a new database is created with **no** data, what state is it in?*
- Every time we ? _____? the database data, we change one state of the database into another state
- **Valid state:** a database state that satisfies the structure and constraints specified in the schema
 - The DBMS is responsible for ensuring that every state of the database is valid
- **Schema evolution:** when the database schema needs to be changed as the application requirements change

Characteristics Of Relations

- **Ordering of Tuples in a Relation**
 - A relation is not sensitive to the ordering of tuples
 - A relation has *no preference* for one tuple ordering over another
- **Ordering of Values within a Tuple**
 - At a more abstract level, the order of attributes and their values is not that important as long as the **correspondence** between attributes and values is maintained
- **Values and NULLs in the Tuples**
 - Each value in a tuple should be **atomic**; thus, composite and multivalued attributes are *not directly* allowed
 - NULL values should be treated carefully during **arithmetic aggregations** or **comparisons** with other values
- **Interpretation (Meaning) of a Relation**
 - The relational model represents facts about both entities and relationships *uniformly* as “relations”
 - **Mapping procedures** exist to convert different constructs of the ER/EER data models into proper relations

Bar	Drinker	Beer
Sue's Bar	Ann	Bud
Joe's Bar	Bob	Miller
Sue's Bar	Cal	Bud Lite
Joe's Bar	Ann	Miller

Bar	Drinker	Beer
Joe's Bar	Ann	Miller
Sue's Bar	Ann	Bud
Joe's Bar	Bob	Miller
Sue's Bar	Cal	Bud Lite

$t = \langle (\text{Name}, \text{Dick Davidson}), (\text{Ssn}, 422-11-2320), (\text{Home_phone}, \text{NULL}), (\text{Address}, 3452 \text{ Elgin Road}), (\text{Office_phone}, (817)749-1253), (\text{Age}, 25), (\text{Gpa}, 3.53) \rangle$

$t = \langle (\text{Address}, 3452 \text{ Elgin Road}), (\text{Name}, \text{Dick Davidson}), (\text{Ssn}, 422-11-2320), (\text{Age}, 25), (\text{Office_phone}, (817)749-1253), (\text{Gpa}, 3.53), (\text{Home_phone}, \text{NULL}) \rangle$

Relational Model Constraints and Relational Database Schemas



Route No.	Miles	Activity
1-95	12	Overlay
1-495	05	
1-66	33	

Relational Model

Activity Code	Activity Name
23	Patching
24	Overlay
25	Crack Sealing

Key = 24

Activity Code	Date	Route No.
24	01/12/01	I-95
24	02/08/01	I-66

Object Model

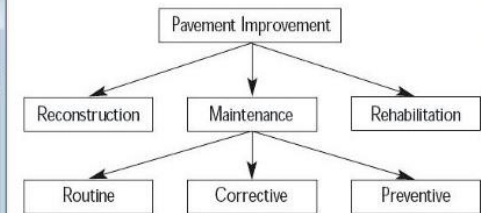
Maintenance Report

Object 1 Instance
01-12-01
24
I-95
2.5
6.0
6.0

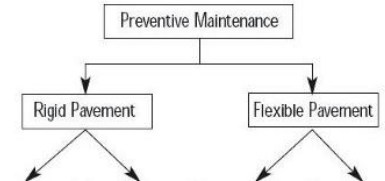
Object 2: Maintenance Activity

Activity Code	
Activity Name	

Network Model



Network Model



Relational Model Constraints

- There are generally many restrictions or **constraints** on the actual values in a database state
- (1) **Inherent model-based/implicit constraints**
 - constraints that are inherent in the data model
 - e.g. the relational model does not allow *duplicate tuples*
- (2) **Schema-based/explicit constraints**
 - constraints that can be directly expressed in the schemas of the data model
 - e.g. domain, key, NULLs, and integrity constraints
- (3) **Application-based constraints**
 - constraints that must be expressed and enforced by the application programs
 - also known as semantic constraints or business rules
 - e.g. constraints that are specified on the *behavior* of particular sets of attributes
 - e.g. “an employee can only work at most 60 hours per week”

Key Constraints

- All tuples in a relation must be *distinct*
 - **translation**: no two tuples can have the **same** combination of values for **all** their attributes
- Such *uniqueness constraint* is achieved via a **key** k of a relation schema R
 - k = any subset of attributes of R for which the uniqueness constraint holds
 - $t_1[k] \neq t_2[k]$ (for *any* two distinct tuples in a relation state $r(R)$)
- **Key** Properties (minimal key):
 - (1) Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key. This *uniqueness* property must be maintained.
 - (2) It is *minimal* –that is, we cannot remove any attributes from the key and still have the uniqueness constraint hold.
 - (3) A key should be *time-invariant* and the uniqueness constraint should hold on *every* valid relation state of the schema, no matter how much new data are being entered.

Key Constraints

- **Superkey** of R :
 - Any *valid* keys of R
 - Can have redundant attributes (i.e., does not need to be minimal)
 - Every relation has a *default superkey* –the set of all its attributes
 - A key is a superkey **but** not vice versa
 - Any superkey formed from a single attribute is also a key
 - *Why? Is this always true? Is it minimal?*

Relation Name

Attributes

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

Tuples

- Can a relation schema have *more than one* key?
- What is the **primary key** of a relation, and how is it *represented* in the schema?

CAR

<u>License_number</u>	Engine_serial_number	Make	Model	Year
Texas ABC-739	A69352	Ford	Mustang	02
Florida TVP-347	B43696	Oldsmobile	Cutlass	05
New York MPO-22	X83554	Oldsmobile	Delta	01
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	04
Texas RSK-629	U028365	Jaguar	XJS	04

Example: The CAR relation, with two candidate keys: License_number and Engine_serial_number.

Key Constraints

- Example: Consider the CAR relation schema:
 - **CAR(State, Reg#, SerialNo, Make, Model, Year)**
 - CAR has two keys:
 - Key1 = {*State, Reg#*}
 - Key2 = {*SerialNo*}
 - Are they both (minimal) keys?
 - Are they both superkeys?

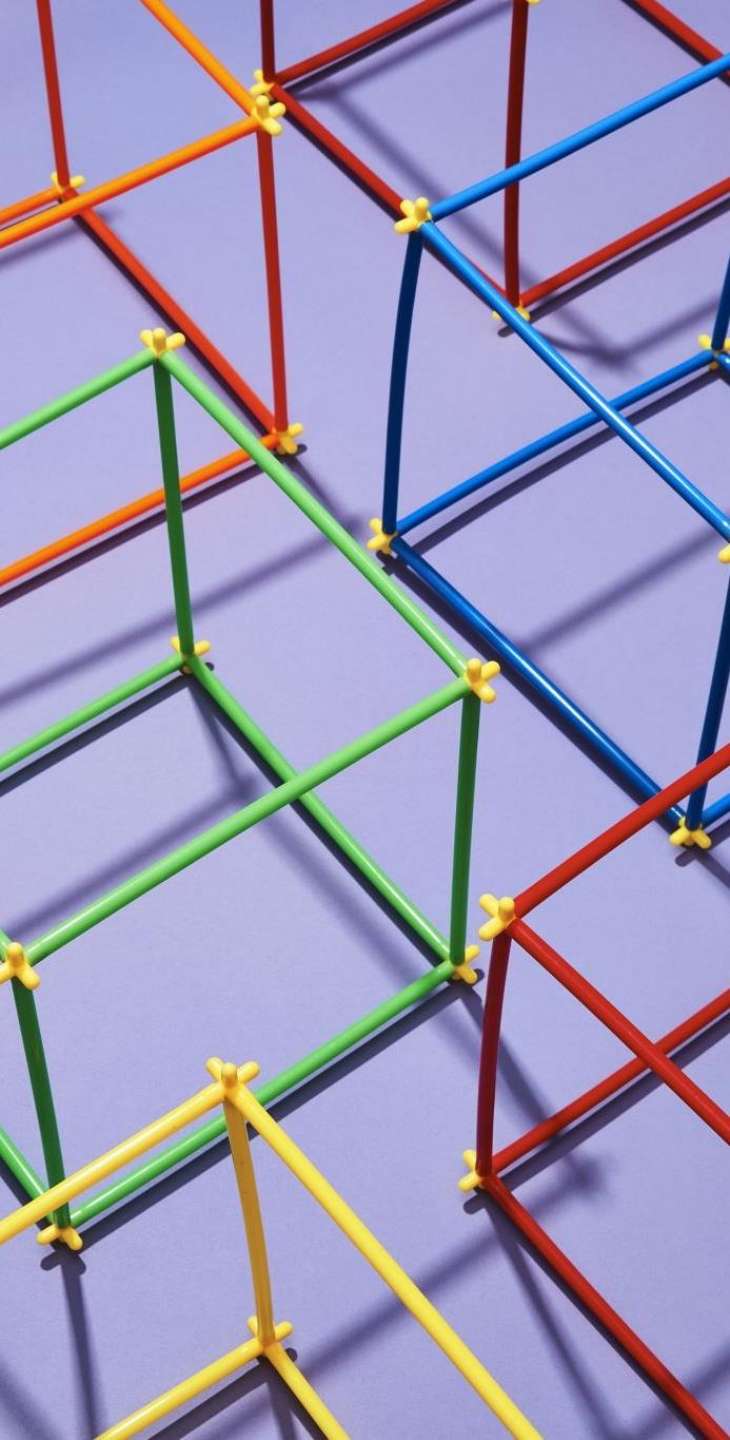


Key Constraints

- Example: Consider the CAR relation schema:
 - **CAR(State, Reg#, SerialNo, Make, Model, Year)**
 - CAR has two keys:
 - Key1 = {State, Reg#}
 - Key2 = {SerialNo}
 - Are they both superkey?
 - Which one is the key (aka: minimal superkey)?
- In general: ✓ (M.K., C.K., P.K.)
 - Any key is always a superkey (but not vice versa)
 - Any set of attributes that *includes a key* is a superkey

Key Constraints

- A relation schema may have multiple valid keys
 - each is called **candidate key**
 - one is chosen arbitrarily to be the **primary key**
 - The **primary key attributes** are underlined
- Example: *CAR(State, Reg#, SerialNo, Make, Model, Year)*
 - Primary key = *SerialNo*
 - *provides unique tuple identity*
- **General Rule**
 - Choose as primary key the smallest of the candidate keys



Relational Databases and Relational Database Schemas

- A relational database usually contains many relations that are related in various ways
- A typical RDBMS uses a *data definition language* (DDL) to define a database schema
 - a feature supported by SQL
- **Relational Database Schema**
 - is a set of relation schemas
 $S = \{R_1, R_2, \dots, R_m\}$ and a set of **integrity constraints** (IC)
- **Relational Database State** (of S):
 - is a set of relation states $DB = \{r_1, r_2, \dots, r_m\}$ such that each r_i is a state of R_i and satisfies IC

← collection of indiv. schemas

no violation in any tuple

Schema Diagram for the COMPANY Relational Database Schema

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



↓ single table

↓ relat.

Entity Integrity, Referential Integrity, and Foreign Keys

- Can a *primary key* take on a NULL value?
- How to maintain *consistency of references* among relation tables that are related?

Entity Integrity Constraint

done (key cons)
NOT NULL constraint
att/column not allow NULL

candidate keys

- Entity Integrity Constraint:
 - No primary key value can be NULL
 - Why?
 - What if the primary key is composed of several attributes?

default:
every att. is allowed to store NULL values



CAR

License number	Engine_serial_number	Make	Model	Year
Texas ABC-739	A69352	NULL	Mustang	02
NULL	B43696	Oldsmobile	Cutlass	05
New York MPO-22	X83554	NULL	Delta	01
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	04
NULL	U028365	Jaguar	XJS	04

NOT NULL

NULL → ambiguous

NULL

?

So far, all constraints are specified within
a single relation.

What if tuples in two relations are related?

Such error should be automatically
detected by the DBMS

Refersal

R1

<u>SID</u>	Name	Address
101	Alice	Rolla
102	Mary	Rolla

typo

R2

<u>SID</u>	<u>Course</u>	Grade
111	CS238	A
101	CS304	A

referencing

relational

data inconsistency issue



Referential Integrity Constraint

- The **referential integrity constraint** is specified between two relations for maintaining *data consistency*
- Definition:**
 - any tuple in a **referencing relation** must refer to an **existing** tuple in the **referenced relation**

DEPARTMENT

<u>Dname</u>	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

PROJECT

<u>Pname</u>	<u>Pnumber</u>	<u>Plocation</u>	<u>Dnum</u>
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	4
Newbenefits	30	Stafford	4

me 10

E.I.C $\rightarrow$ PK
R.I.C $\rightarrow$ FK

Foreign Keys

- A *foreign key* is commonly used to specify a referential integrity constraint between two relation schemas

(referencing relation)

- A set of attributes FK in relation schema R_1 is a **foreign key** of R_1 that **references** relation schema R_2 if and only if:

- The attributes in FK have the same domain(s) as the primary key attributes PK of R_2
- All references conform to the relationship $t_i[FK] = t_j[FK]$ where t_i is a tuple in $r_i(R_1)$ and t_j is a tuple in $r_j(R_2)$

data type

value exist in referenced relation / CK

o FK should referencing the PK / CK in the referenced relation

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

Foreign Keys

Employee: PK (SSN)

FK?

FK #1 (Dno) references

PK (Dnumber) in Department

FK #2 (Super_ssn) references

PK (SSN) in Employee

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	NULL	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
NULL	Stafford
5	Bellaire
5	Sugarland
5	Houston

dom. datatype

Department:
PK (Dnumber)
FK (Mgr_ssn) references

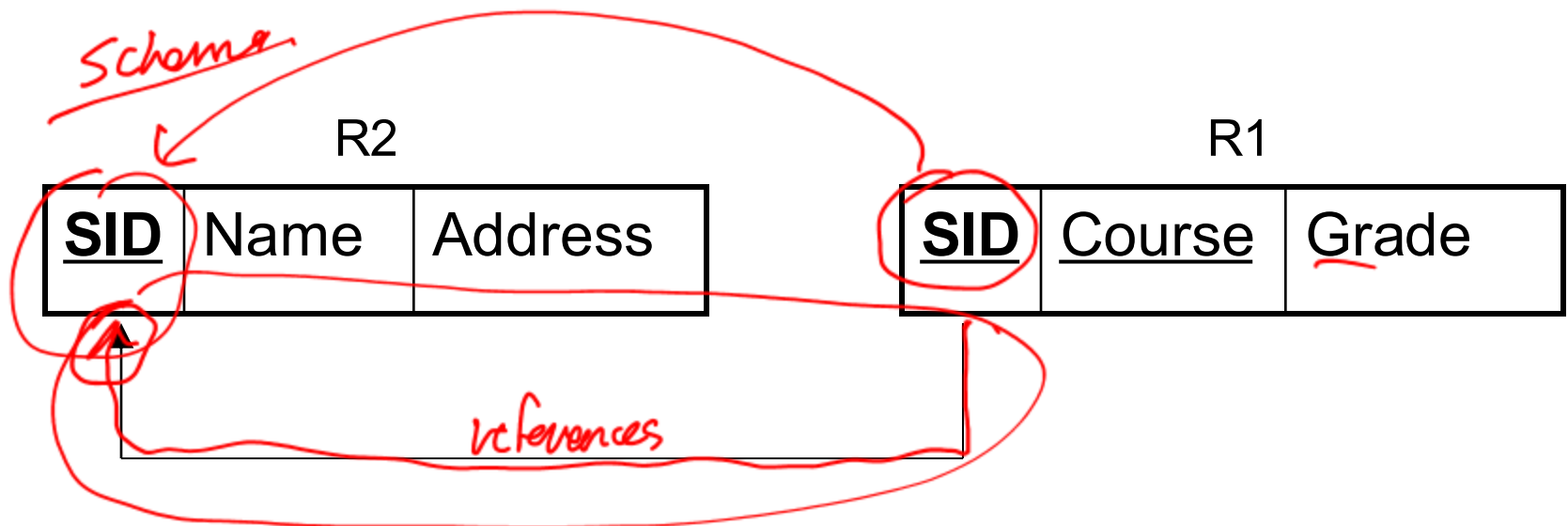
valid existing
PK (SSN) in Employee

PK (Dnumber, Dlocation)
FK (Dnumber) references PK (Dn) in Department

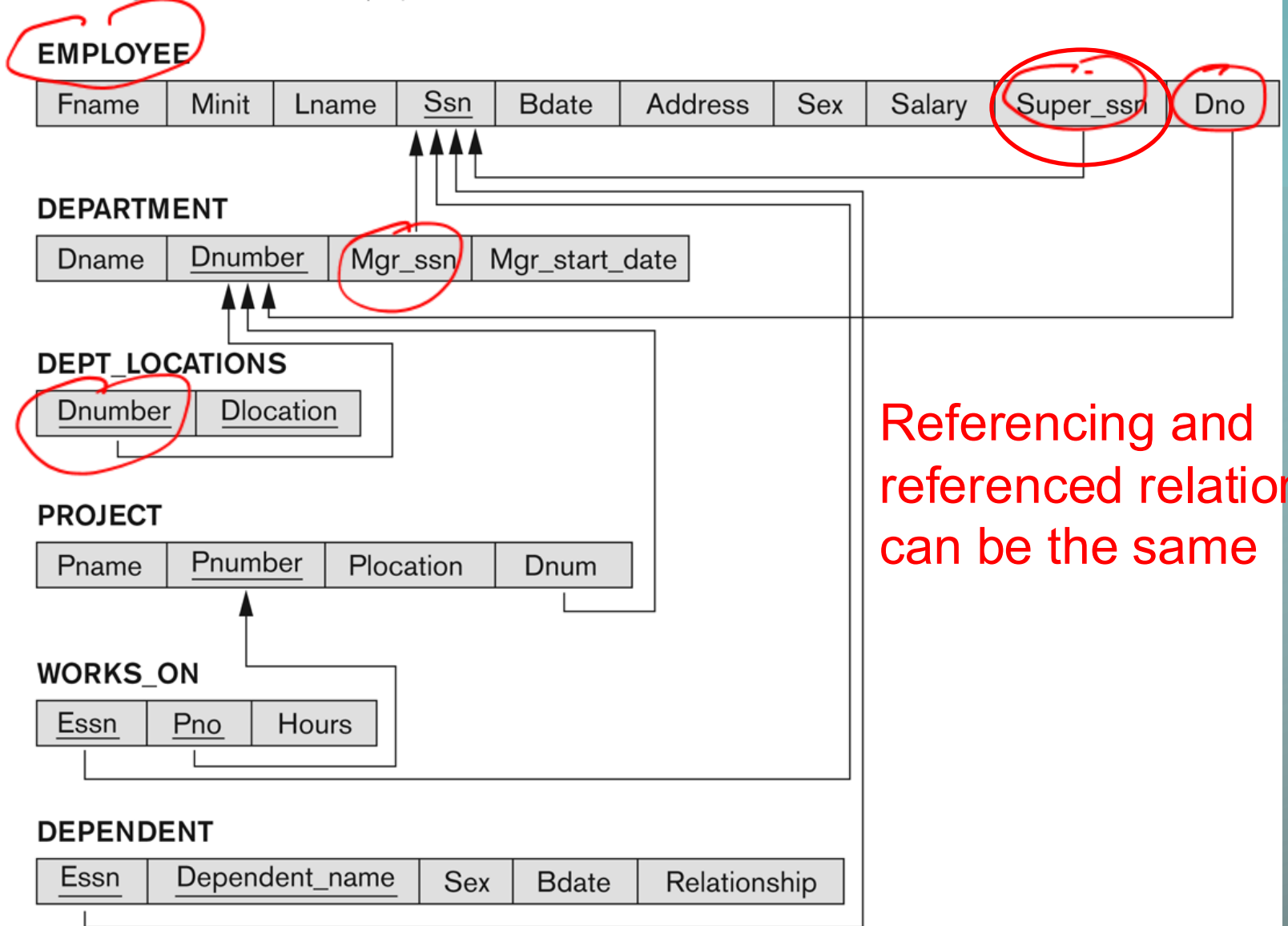
Comp. PK

Referential Integrity

A referential integrity constraint can be displayed in a relational database schema as **a directed arc** from R1.FK to R2.



Referential integrity constraints displayed on the COMPANY relational database schema.



Referencing and referenced relations can be the same

Other Types of Constraints

- **Semantic Integrity Constraints:**
 - based on application semantics and cannot be expressed by the relational model inherently.
 - Example:
 - “an employee can only work at most 60 hours per week”
 - “the salary of an employee should not exceed the salary of their supervisor”
 - Can be enforced within application programs or using a constraint specification language
 - E.g., TRIGGERS and ASSERTIONS in SQL
- **State Constraints:** Primary/Foreign Keys, etc.

Exercise

Consider the following relations for a database that keeps track of student enrollment in courses and the books adopted for each course:

STUDENT(SSN, Name, Major, Bdate)

COURSE(Course#, Cname, Dept)

ENROLL(SSN, Course#, Quarter, Grade)

BOOK_ADOPTION(Course#, Quarter, Book_ISBN)

TEXT(Book_ISBN, Book_Title, Publisher, Author)

Draw a relational schema diagram specifying the foreign keys for this schema.

Operations of the Relational Model

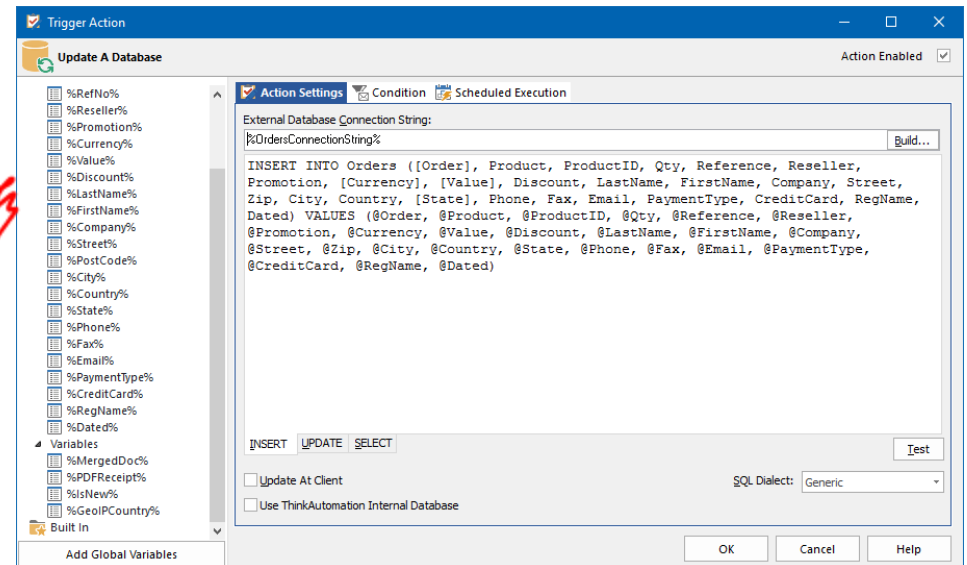
- The operations of the relational model can be categorized into:

- Retrieval
 - Query a database
- Updates
 - Change the database

- Basic update operations for changing the database:

- **INSERT** a new tuple in a relation
- **DELETE** an existing tuple from a relation
- **MODIFY** an attribute of an existing tuple

*no integrity
conf.
violation*



Possible Violations for Insert Operation

→ Action: Rejection

- **INSERT** may violate any of the constraints:
 - Domain constraint / NOT NULL constraint:
 - if one of the attribute values provided for the new tuple is not of the specified attribute domain or it is NULL when not allowed
 - Key constraint:
 - if the value of a key attribute in the new tuple already exists in another tuple in the relation
 - Entity integrity:
 - if the primary key value is NULL in the new tuple
 - Referential integrity:
 - if a foreign key value in the new tuple references a primary key value that does not exist in the referenced relation

The Insert Operation

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Houston, TX	M	39000	888665555	5
Joyce	A	English	453453453	1972-07-31	5631 Rife, Houston, TX	F	26000	888665555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dahlia, Houston, TX	M	25000	888665555	5
James	E	Borg	888665555	1937-11-10	450 St. Louis, Houston, TX	M	29000	888665555	5

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

The default option for an insertion that **violates** any constraints is to **reject** it.

■ Operation:

Insert <'Cecilia', 'F', 'Kolonsky', NULL, '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.

Result: This insertion violates the entity integrity constraint (NULL for the primary key Ssn), so it is rejected.

■ Operation:

Insert <'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, '987654321', 4> into EMPLOYEE.

Result: This insertion violates the key constraint because another tuple with the same Ssn value already exists in the EMPLOYEE relation, and so it is rejected.

■ Operation:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windswept, Katy, TX', F, 28000, '987654321', 7> into EMPLOYEE.

Result: This insertion violates the referential integrity constraint specified on Dno in EMPLOYEE because no corresponding referenced tuple exists in DEPARTMENT with Dnumber = 7.

■ Operation:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.

Result: This insertion satisfies all constraints, so it is acceptable.

Possible Violations for Delete Operation

- DELETE** may violate only referential integrity:

– Can be remedied by several actions

Cascade
Key c.? X
G.I.L X
tuple deleted, what if other tuples referencing it?
→ PK that ref. FK in db.

S

<u>sid</u>	sname	year
240	white	3
202	jones	3
450	adams	1

R

<u>tid</u>	<u>sid</u>	<u>course</u>
20	202	os
10	450	calculus
20	202	db
20	240	db

?

no cascade

The Delete Operation

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

■ **Operation:**

Delete the WORKS_ON tuple with Essn = '999887777' and Pno = 10.
Result: This deletion is acceptable and deletes exactly one tuple.

■ **Operation:**

Delete the EMPLOYEE tuple with Ssn = '999887777'.

Result: This deletion is not acceptable, because there are tuples in WORKS_ON that refer to this tuple. Hence, if the tuple in EMPLOYEE is deleted, referential integrity violations will result.

■ **Operation:**

Delete the EMPLOYEE tuple with Ssn = '333445555'.

Result: This deletion will result in even worse referential integrity violations, because the tuple involved is referenced by tuples from the EMPLOYEE, DEPARTMENT, WORKS_ON, and DEPENDENT relations.

1. **Restrict** – *reject the deletion*
2. **Cascade** – *propagate the deletion by deleting tuples that reference the tuple that is being deleted*
3. **Set null/default** – *modify the deletion by modifying the referencing attribute values of the tuples that reference the tuple that is being deleted; the values can be set to NULL or changed to reference another (default) valid tuple*

The Update Operation

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

- **Operation:**
Update the salary of the EMPLOYEE tuple with Ssn = '999887777' to 28000.
Result: Acceptable.
- **Operation:**
Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 1.
Result: Acceptable.
- **Operation:**
Update the Dno of the EMPLOYEE tuple with Ssn = '999887777' to 7.
Result: Unacceptable, because it violates referential integrity.
- **Operation:**
Update the Ssn of the EMPLOYEE tuple with Ssn = '999887777' to '987654321'.
Result: Unacceptable, because it violates primary key constraint by repeating a value that already exists as a primary key in another tuple; it violates referential integrity constraints because there are other relations that refer to the existing value of Ssn.

- Special attentions must be paid to updating attributes that are part of a _____ or a _____.

Three-Schema Architecture

Goal:

- Separate the *user applications* from the *physical database*

External level:

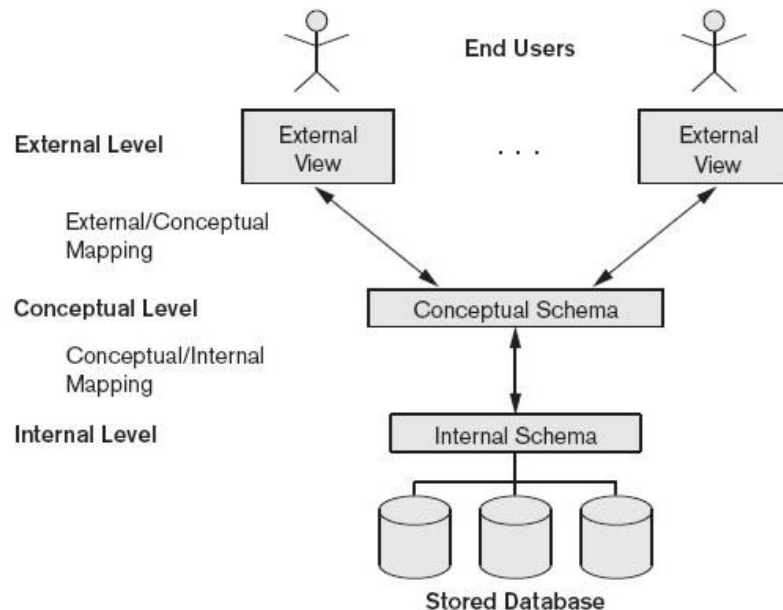
- Includes a number of *user views*: the part of the database that a particular user group is allowed to access.

Conceptual level:

- Describes the structure of the whole database (e.g., entities, relationships, user operations, constraints, etc.)

Internal level:

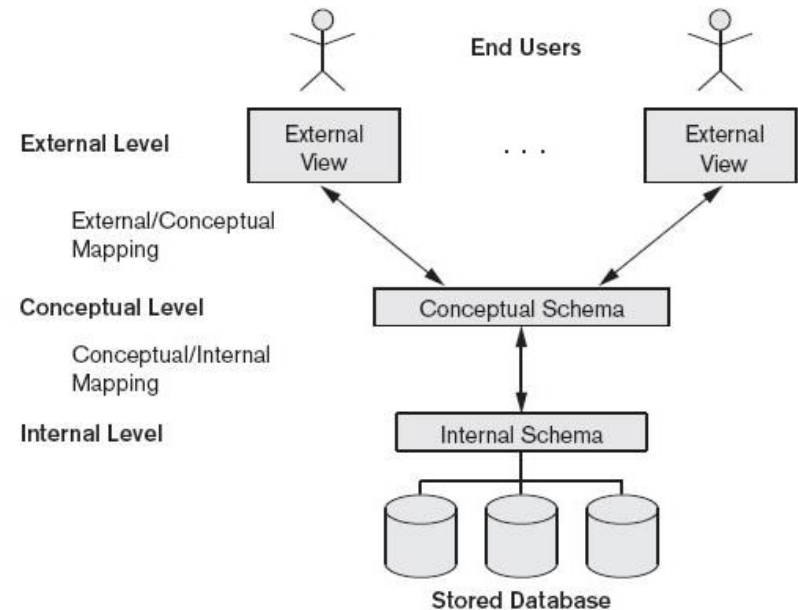
- Describes the physical storage structure of the database (e.g., file structure/index usage).



Three-Schema Architecture

1. Self-describing nature using a schema catalog
2. Insulation of programs and data
3. Support of multiple user views

- **Data independence** in the three-schema architecture
 - The capacity to change the schema at one level of a database system without having to change the schema at the next **higher** level



Data Independence

- **Logical Data Independence**
 - The capacity to change the conceptual schema without having to change the external schemas and their associated application programs.
 - For example:
 - Database expansion & reduction
 - Adding a record type or data item
 - Removing a record type or data item
 - Only the view definition and the mappings need to be changed
 - **mappings**: the process of transforming requests and results between levels
 - Database reorganization
 - Constraints changes
 - Restructuring of schema constructs
 - should not affect the external schemas

Database Expansion

Easy to achieve?

Update the *view definitions* or *mappings* in the external schema (application interface) or application programs.

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

GRADE_REPORT



Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

- Data queries and retrievals *depend upon the logical structure of data*
- Addition of attributes, entities, or relationships
- Deletion of attributes, entities, or relationships

Database Reduction

Easy to achieve?

- LDI can be achieved by ensuring that the external schemas that refer only to the **remaining data** are not affected.

Update the **view definitions** or **mappings** in the external schema (application interface) or application programs.

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

- Data queries and retrievals *depend upon the logical structure of data*
- Addition of attributes, entities, or relationships
- Deletion of attributes, entities, or relationships

Database Reorganization

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

Update the *view definitions* or *mappings* in the external schema (application interface) or application programs.

- **Physical Data Independence**

- The capacity to change the internal schema without having to change the conceptual schema.
- For example:
 - Reorganization of some physical files
 - Creating additional access structures
- Which one is harder to achieve, logical or physical data independence?

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone



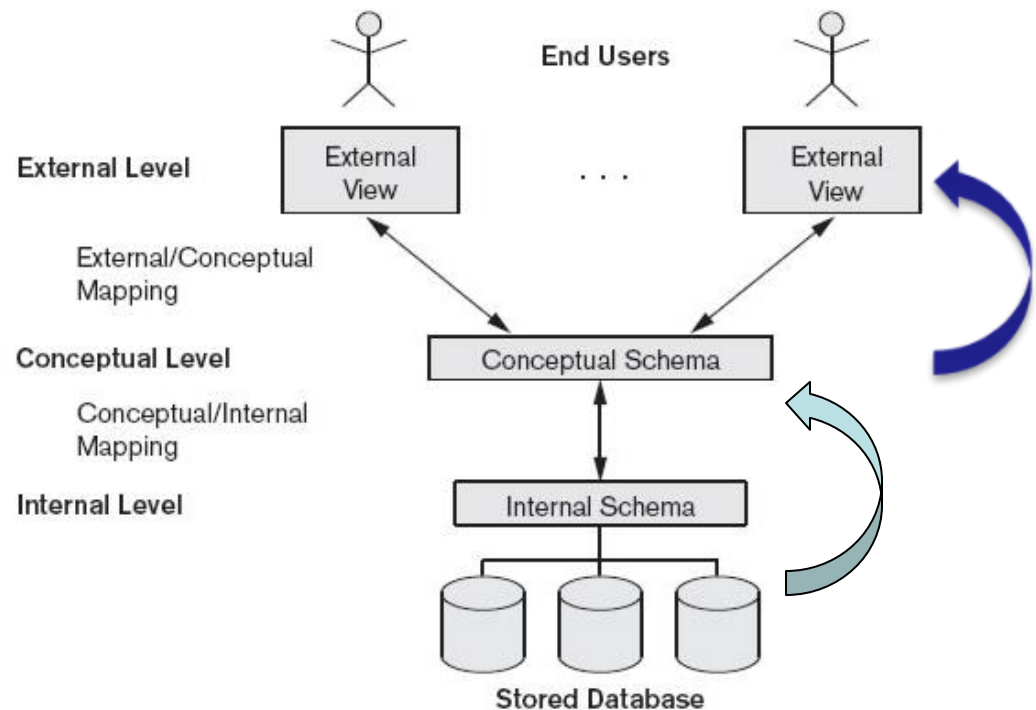
Access path (1)
section_identifier



Access path (2)
semester and year

Data Independence

- In a DBMS that implements the multi-level architecture, its **catalog** contains information on how to **map** requests and data among the various tiers.
- Data independence can be achieved when any changes made in a specific tier only requires the **mapping** to the next higher level to be changed.
- Constant changes to the mappings could contribute **overheads** during the compilation or execution of a query or program.



Summary of Lecture 5

The Relational Data Model and Relational Database Constraints



- **What we learned?**
 - Relational Model Concepts
 - Domains, Attributes, Tuples, and Relations
 - Relational Model Notations
 - Relational Model Constraints
 - Relational Database Schemas
 - Entity Integrity, Referential Integrity, and Foreign Keys
 - Update Operations and Dealing with Constraint Violations
- **What you understood the most?**
- **What are the most difficult to you?**